Лабораторна робота № 3

Діагностика шкал вимірювання

Курс СК26 «Інтелектуальний аналіз та візуалізація даних»

Тривалість: 2 год аудиторно + 3 год самостійно · **Оцінка:** 4 бали **Файли:** `Ковід_дані_навчальні.xlsx` , `ЛР3_помилковий_аналіз.ipynb`

Шкала вимірювання визначає, які операції над змінною взагалі мають сенс. Комп'ютер цього не знає: він порахує середнє від номера телефону й не поскаржиться. Знати має вимірювач.

У цій роботі ви складаєте **паспорт восьми змінних** свого варіанта, показуєте на власних даних, які статистики для них недопустимі, і виправляєте чужий аналіз, де таких помилок десять.

In [3]:
```
ПРІЗВИЩЕ_ІМЯ = 'Яцишин Роман'      # напишіть тут своє прізвище та ім'я
ГРУПА        = 'ШІ-32'
НОМЕР        = 19       # ваш номер за списком групи (1–30)

assert ПРІЗВИЩЕ_ІМЯ and ГРУПА and НОМЕР, 'Заповніть три поля вище'
assert 1 <= НОМЕР <= 30, 'Номер має бути від 1 до 30'
```

In [4]:
```
import json
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_colwidth', 60)

ВАРІАНТИ = json.load(open('lr3_variants_student.json', encoding='utf-8'))
В = ВАРІАНТИ[НОМЕР - 1]

print('=' * 74)
print(f'ВАРІАНТ {В["варіант"]}   ·   {ПРІЗВИЩЕ_ІМЯ}   ·   {ГРУПА}')
print('=' * 74)
print()
print('Ваші вісім стовпців:')
for i, c in enumerate(В['стовпці'], 1):
    print(f'   {i}. {c}')
print()
print('Ваша пара для аналізу зв’язку:', ' та '.join(В['пара']))
print()
```

```
print('Ваше додаткове питання:')
print('  ', B['питання'])
```

```
================================================================================
ВАРІАНТ 19   ·   Яцишин Роман   ·   ШІ-32
================================================================================

Ваші вісім стовпців:
   1. номер і/х
   2. Стать
   3. ЛН
   4. Виписка
   5. Днів до поступ
   6. Т 37-37,5
   7. Гемогл2
   8. Прот час1

Ваша пара для аналізу зв’язку: spO2 без дот та помер/жив

Ваше додаткове питання:
   Знайдіть у файлі стовпець, у якому порожня клітинка означає не «невідомо», а «нор
ма». Доведіть це числами й поясніть, чому dropna() тут небезпечний.
```

In [5]:
```
ФАЙЛ = 'Ковід_дані_навчальні.xlsx'
df = pd.read_excel(ФАЙЛ, sheet_name='Лист1', engine='openpyxl')
МОЇ = B['стовпці']

print('Розмір набору:', df.shape)
df[МОЇ].head(8)
```

```
Розмір набору: (121, 77)
```

Out[5]:

	номер і/х	Стать	ЛН	Виписка	Днів до поступ	Т 37-37,5	Гемогл2	Прот час1
0	2377/239	ч	III-IV	2022-03-10 00:00:00	5	1900-01-11 00:00:00	144	26.4
1	1121/19	ч	II-III	2022-03-16 00:00:00	12	19	137	18.0
2	283/674	ж	II	2022-01-18 00:00:00	15	17	105	10.6
3	2372/513	ж	III-IV	2022-01-13 00:00:00	15	NaN	116	18.3
4	1441/671	ж	II-III	2022-03-11 00:00:00	7	4	124	11.8
5	2573/354	ж	II-III	2021-12-26 00:00:00	8(4дні лікув в Садгорі)	9	120	9.1
6	1762/555	ч	II-III	2022-05-11 00:00:00	7	4	151	11.3
7	2664/348	ж	I	2022-05-28 00:00:00	9	3	137	9.5

Етап 1. Паспорт змінних

Для кожного з восьми стовпців визначте шкалу вимірювання. Спирайтеся не на тип даних у pandas, а на **зміст величини**: що означають її значення і які порівняння між ними мають сенс.

Нагадування:

Шкала	Ознака
Номінальна	значення можна лише порівняти на рівність
Порядкова	є природний порядок, але відстані між сусідніми рівнями невідомі
Інтервальна	різниці мають сенс, нуль умовний
Відношень	є природний нуль, відношення мають сенс
Абсолютна	природні і нуль, і одиниця вимірювання

Окремо позначайте **ідентифікатори** — значення, що не є величиною взагалі (номер історії хвороби, прізвище, телефон).

In [6]:
```
# Спершу подивіться, що каже pandas — і не довіряйте йому
df[МОЇ].info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 121 entries, 0 to 120
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   номер і/х       121 non-null    object 
 1   Стать           121 non-null    str    
 2   ЛН              121 non-null    str    
 3   Виписка         120 non-null    object 
 4   Днів до поступ  120 non-null    object 
 5   Т 37-37,5       66 non-null     object 
 6   Гемогл2         105 non-null    object 
 7   Прот час1       112 non-null    float64
dtypes: float64(1), object(5), str(2)
memory usage: 7.7+ KB
```

In [7]:
```
# Далі — що є всередині насправді
for c in МОЇ:
    з = df[c].dropna()
    типи = ', '.join(f'{t.__name__}×{n}' for t, n in з.map(type).value_counts().ite
    print(f'--- {c}')
    print(f'    непорожніх {len(з)} з {len(df)} | унікальних {з.nunique()} | типи:
    print(f'    приклади: {[repr(v) for v in з.unique()[:4]]}')
    print()
```

```
--- номер і/х
    непорожніх 121 з 121 | унікальних 119 | типи: int×65, str×56
    приклади: ["'2377/239'", "'1121/19'", "'283/674'", "'2372/513'"]

--- Стать
    непорожніх 121 з 121 | унікальних 4 | типи: str×121
    приклади: ["'ч'", "'ж'", "'чол'", "'жін'"]

--- ЛН
    непорожніх 121 з 121 | унікальних 6 | типи: str×121
    приклади: ["'III-IV'", "'II-III'", "'II'", "'I'"]

--- Виписка
    непорожніх 120 з 121 | унікальних 90 | типи: datetime×118, str×2
    приклади: ['datetime.datetime(2022, 3, 10, 0, 0)', 'datetime.datetime(2022, 3, 1
6, 0, 0)', 'datetime.datetime(2022, 1, 18, 0, 0)', 'datetime.datetime(2022, 1, 13,
0, 0)']

--- Днів до поступ
    непорожніх 120 з 121 | унікальних 15 | типи: int×119, str×1
    приклади: ['5', '12', '15', '7']

--- Т 37-37,5
    непорожніх 66 з 121 | унікальних 29 | типи: str×38, int×27, datetime×1
    приклади: ['datetime.datetime(1900, 1, 11, 0, 0)', '19', '17', '4']

--- Гемогл2
    непорожніх 105 з 121 | унікальних 60 | типи: int×104, str×1
    приклади: ['144', '137', '105', '116']

--- Прот час1
    непорожніх 112 з 121 | унікальних 63 | типи: float×112
    приклади: ['np.float64(26.4)', 'np.float64(18.0)', 'np.float64(10.6)', 'np.float
64(18.3)']
```

Заповніть паспорт

Для кожного стовпця вкажіть шкалу та **обґрунтування** — одне речення про те, чому саме ця шкала. Обґрунтування оцінюється, назва шкали сама по собі — ні.

In [8]:
```python
ПАСПОРТ = {
    # 'назва стовпця': ('шкала', 'обґрунтування однією фразою'),
    'номер і/х':      ('ідентифікатор', 'номер історії хвороби лише розрізняє пацієн
    'Стать':          ('номінальна',    'значення «ж» і «ч» можна лише порівняти на
    'ЛН':             ('порядкова',     'ступінь легеневої недостатності I < II < I
    'Виписка':        ('інтервальна',   'дата виписки: різниця дат має сенс (кількі
    'Днів до поступ': ('відношень',     'тривалість у днях має природний нуль, і «1
    'Т 37-37,5':      ('відношень',     'кількість днів із температурою 37–37,5 (ну
    'Гемогл2':        ('відношень',     'концентрація гемоглобіну в г/л має природн
    'Прот час1':      ('відношень',     'протромбіновий час у секундах має природни
}

for c in МОЇ:
    ПАСПОРТ.setdefault(c, ('', ''))
```

```
паспорт_df = pd.DataFrame(
    [(c, ПАСПОРТ[c][0], ПАСПОРТ[c][1]) for c in МОЇ],
    columns=['Стовпець', 'Шкала', 'Обґрунтування'])
паспорт_df
```

Out[8]:

	Стовпець	Шкала	Обґрунтування
0	номер і/х	ідентифікатор	номер історії хвороби лише розрізняє пацієнтів і не є ве...
1	Стать	номінальна	значення «ж» і «ч» можна лише порівняти на рівність, пор...
2	ЛН	порядкова	ступінь легеневої недостатності I < II < III < IV має пр...
3	Виписка	інтервальна	дата виписки: різниця дат має сенс (кількість днів), а н...
4	Днів до поступ	відношень	тривалість у днях має природний нуль, і «10 днів у два р...
5	Т 37-37,5	відношень	кількість днів із температурою 37–37,5 (нуль днів природ...
6	Гемогл2	відношень	концентрація гемоглобіну в г/л має природний нуль, відно...
7	Прот час1	відношень	протромбіновий час у секундах має природний нуль, віднош...

In [9]:
```
# --- САМОПЕРЕВІРКА 1 ---
ДОПУСТИМІ = {'номінальна', 'порядкова', 'інтервальна', 'відношень',
             'абсолютна', 'ідентифікатор'}
for c in МОЇ:
    ш, об = ПАСПОРТ[c]
    assert ш.strip().lower() in ДОПУСТИМІ, f'{c}: шкалу «{ш}» не розпізнано'
    assert len(об.strip()) >= 25, f'{c}: обґрунтування закоротке'
print('Самоперевірка 1 пройдена: усі вісім стовпців мають шкалу та обґрунтування.')
```

```
Самоперевірка 1 пройдена: усі вісім стовпців мають шкалу та обґрунтування.
```

Етап 2. Допустимі статистики

Для кожного стовпця обчисліть **тільки те, що дозволяє його шкала**.

- номінальна та ідентифікатор — мода, частоти
- порядкова — додатково медіана та квантилі
- інтервальна — додатково середнє та стандартне відхилення
- відношень і абсолютна — усе, зокрема коефіцієнт варіації

Якщо для стовпця не можна порахувати нічого, крім частот, — так і напишіть.

In [10]:

```python
import re
import datetime

# --- функції очищення «брудних» клітинок (використовуються далі в усій роботі) ---
SEX_MAP = {'ж': 'ж', 'жін': 'ж', 'ч': 'ч', 'чол': 'ч'}          # 4 написання -> 2
LN_ORDER = ['I', 'I-II', 'II', 'II-III', 'III', 'III-IV']       # природний порядок


def to_num(series):
    # '2 дні в стац' -> 2, 'немає' -> NaN, дата 1900-01-11 (Excel сприйняв число 11
    def conv(v):
        if v is None or (isinstance(v, float) and np.isnan(v)):
            return np.nan
        if isinstance(v, datetime.datetime):
            return float(v.timetuple().tm_yday) if v.year == 1900 else np.nan
        if isinstance(v, (int, float, np.number)):
            return float(v)
        m = re.search(r'\d+(?:[.,]\d+)?', str(v))
        return float(m.group().replace(',', '.')) if m else np.nan
    return series.map(conv)


def clean_ln(series):
    # у файлі змішано кирилицю й латиницю (III-IV): приводимо все до латинських I,
    s = series.astype(str).str.strip().str.replace('\u0406', 'I')
    return pd.Series(pd.Categorical(s, categories=LN_ORDER, ordered=True), index=se


def get_clean(col):
    s = df[col]
    if col == 'Стать':
        return s.astype(str).str.strip().map(SEX_MAP)
    if col == 'ЛН':
        return clean_ln(s)
    if col == 'Виписка':
        return pd.to_datetime(s, errors='coerce')               # хибні дати '24.02
    if col == 'номер і/х':
        return s.astype(str).str.strip()
    return to_num(s)


def допустима_статистика(стовпець, шкала):
    """Повертає словник статистик, дозволених для цієї шкали."""
    з = get_clean(стовпець).dropna()
    р = {'непорожніх': len(з), 'унікальних': з.nunique()}
    ш = шкала.strip().lower()

    if ш in ('ідентифікатор',):
        return р
    # номінальна та вище: мода й частоти
    р['мода'] = з.mode().iloc[0]
    р['частоти'] = з.value_counts().head(3).to_dict()
    рівень = {'номінальна': 1, 'порядкова': 2, 'інтервальна': 3, 'відношень': 4, 'а

    if рівень >= 2:                                             # порядкова: медіан
```

```
        if isinstance(з.dtype, pd.CategoricalDtype):
            коди = з.cat.codes.astype(float)
            for ім, q in (('медіана', .5), ('q25', .25), ('q75', .75)):
                p[ім] = з.cat.categories[int(np.quantile(коди, q, method='lower'))]
        else:
            p['медіана'] = з.median()
            p['q25'], p['q75'] = з.quantile(.25), з.quantile(.75)
    if рівень >= 3:                                          # інтервальна: сере
        if pd.api.types.is_datetime64_any_dtype(з):
            p['середнє'] = з.mean().date()
            p['ст. відхилення, днів'] = round(з.std() / pd.Timedelta(days=1), 1)
        else:
            p['середнє'], p['ст. відхилення'] = round(з.mean(), 3), round(з.std(),
    if рівень >= 4:                                          # відношень: коефіц
        p['коефіцієнт варіації, %'] = round(100 * з.std() / з.mean(), 1)
    return p


for c in МОЇ:
    print(f'--- {c}  ({ПАСПОРТ[c][0]})')
    print('   ', допустима_статистика(c, ПАСПОРТ[c][0]))
    print()
```

```
--- номер і/х  (ідентифікатор)
    {'непорожніх': 121, 'унікальних': 119}

--- Стать  (номінальна)
    {'непорожніх': 121, 'унікальних': 2, 'мода': 'ж', 'частоти': {'ж': 75, 'ч': 46}}

--- ЛН  (порядкова)
    {'непорожніх': 121, 'унікальних': 6, 'мода': 'II-III', 'частоти': {'II-III': 59,
'II': 21, 'III': 17}, 'медіана': 'II-III', 'q25': 'II', 'q75': 'II-III'}

--- Виписка  (інтервальна)
    {'непорожніх': 118, 'унікальних': 88, 'мода': Timestamp('2021-09-07 00:00:00'),
'частоти': {Timestamp('2021-11-19 00:00:00'): 3, Timestamp('2021-09-07 00:00:00'):
3, Timestamp('2021-09-15 00:00:00'): 3}, 'медіана': Timestamp('2022-03-15 00:00:0
0'), 'q25': Timestamp('2021-11-16 18:00:00'), 'q75': Timestamp('2022-04-27 18:00:0
0'), 'середнє': datetime.date(2022, 1, 29), 'ст. відхилення, днів': 109.5}

--- Днів до поступ  (відношень)
    {'непорожніх': 120, 'унікальних': 14, 'мода': np.float64(7.0), 'частоти': {7.0:
33, 10.0: 21, 5.0: 13}, 'медіана': np.float64(7.0), 'q25': np.float64(6.0), 'q75': n
p.float64(10.0), 'середнє': np.float64(8.417), 'ст. відхилення': np.float64(3.847),
'коефіцієнт варіації, %': np.float64(45.7)}

--- Т 37-37,5  (відношень)
    {'непорожніх': 65, 'унікальних': 16, 'мода': np.float64(2.0), 'частоти': {2.0: 1
7, 4.0: 11, 3.0: 11}, 'медіана': np.float64(3.0), 'q25': np.float64(2.0), 'q75': np.
float64(5.0), 'середнє': np.float64(4.692), 'ст. відхилення': np.float64(3.913), 'ко
ефіцієнт варіації, %': np.float64(83.4)}

--- Гемогл2  (відношень)
    {'непорожніх': 104, 'унікальних': 59, 'мода': np.float64(142.0), 'частоти': {14
4.0: 5, 142.0: 5, 137.0: 4}, 'медіана': np.float64(134.0), 'q25': np.float64(116.0),
'q75': np.float64(144.0), 'середнє': np.float64(128.712), 'ст. відхилення': np.float
64(23.514), 'коефіцієнт варіації, %': np.float64(18.3)}

--- Прот час1  (відношень)
    {'непорожніх': 112, 'унікальних': 63, 'мода': np.float64(18.0), 'частоти': {18.
0: 6, 11.8: 5, 13.2: 5}, 'медіана': np.float64(12.9), 'q25': np.float64(11.1), 'q7
5': np.float64(16.125), 'середнє': np.float64(14.298), 'ст. відхилення': np.float64
(4.582), 'коефіцієнт варіації, %': np.float64(32.0)}
```

Етап 3. Три безглузді числа

`df.describe()` порахує статистики для всього, що схоже на число. Частина цих чисел не має жодного змісту.

Знайдіть **щонайменше три** такі числа у своєму наборі: обчисліть їх, а потім поясніть, чому кожне з них безглузде.

In [11]:
```
df[МОЇ].describe(include='all').T
```

Out[11]:

	count	unique	top	freq	mean	std	min	25%	50%	75%	ma
номер і/х	121.0	119.0	1837.0	2.0	NaN	NaN	NaN	NaN	NaN	NaN	Na
Стать	121	4	ж	56	NaN	NaN	NaN	NaN	NaN	NaN	Na
ЛН	121	6	II-III	59	NaN	NaN	NaN	NaN	NaN	NaN	Na
Виписка	120	90	2021-11-19 00:00:00	3	NaN	NaN	NaN	NaN	NaN	NaN	Na
Днів до поступ	120.0	15.0	7.0	33.0	NaN	NaN	NaN	NaN	NaN	NaN	Na
Т 37-37,5	66	29	2 дні в стац	9	NaN	NaN	NaN	NaN	NaN	NaN	Na
Гемогл2	105.0	60.0	144.0	5.0	NaN	NaN	NaN	NaN	NaN	NaN	Na
Прот час1	112.0	NaN	NaN	NaN	14.298214	4.582477	7.9	11.1	12.9	16.125	33

In [12]:
```
# числа, які порахує describe() або необачний аналітик, і які нічого не означають
id_num = pd.to_numeric(df['номер і/х'], errors='coerce')                # номери, що
sex_code = get_clean('Стать').map({'ч': 0, 'ж': 1})
ln_code = get_clean('ЛН').cat.codes + 1                                 # I=1 ... II
dates = get_clean('Виписка').dropna()
days_1970 = (dates - pd.Timestamp('1970-01-01')).dt.days
days_2000 = (dates - pd.Timestamp('2000-01-01')).dt.days
cv_1970 = 100 * days_1970.std() / days_1970.mean()
cv_2000 = 100 * days_2000.std() / days_2000.mean()

БЕЗГЛУЗДІ = [
    # ('стовпець', 'яка статистика', значення, 'чому це число нічого не означає'),
    ('номер і/х', 'середнє', round(id_num.mean(), 1),
     'Це номер історії хвороби, а не величина: середнє номерів не описує ні пацієнт
    ('Стать (код ч=0, ж=1)', 'середнє', round(sex_code.mean(), 3),
     f'Стать номінальна, коди 0 і 1 довільні: при кодуванні ж=0, ч=1 «середня стать
    ('ЛН (коди 1–6)', 'середнє', round(ln_code.mean(), 2),
     f'ЛН порядкова, відстані між ступенями невідомі: при кодах 1, 2, 4, 8, 16, 32
    ('Виписка', 'коефіцієнт варіації', round(cv_1970, 3),
     f'У дат нуль умовний: від 1970 року CV = {cv_1970:.3f}%, а від 2000 року CV =
]

assert len(БЕЗГЛУЗДІ) >= 3, 'Потрібно щонайменше три приклади'
pd.DataFrame(БЕЗГЛУЗДІ, columns=['Стовпець', 'Статистика', 'Значення', 'Чому безглу
```

Out[12]:

	Стовпець	Статистика	Значення	Чому безглуздe
0	номер і/х	середнє	1770.100	Це номер історії хвороби, а не величина: середнє номерів...
1	Стать (код ч=0, ж=1)	середнє	0.620	Стать номінальна, коди 0 і 1 довільні: при кодуванні ж=0...
2	ЛН (коди 1–6)	середнє	3.570	ЛН порядкова, відстані між ступенями невідомі: при кодах...
3	Виписка	коефіцієнт варіації	0.576	У дат нуль умовний: від 1970 року CV = 0.576%, а від 200...

Етап 4. Міра зв'язку для вашої пари

Для двох змінних вашого варіанта треба оцінити зв'язок. Міру обирає **пара шкал**, а не зручність:

Поєднання шкал	Міра
відношень × відношень	кореляція Пірсона
порядкова × порядкова	рангова кореляція Спірмена
номінальна × номінальна	χ^2 та коефіцієнт Крамера (V)
відношень × бінарна номінальна	точковий бісеріальний коефіцієнт
номінальна × порядкова	коефіцієнт Крамера (V)

Обґрунтуйте вибір, обчисліть міру й поясніть результат. Окремо покажіть, **що станеться**, якщо застосувати Пірсона там, де він недопустимий, — і чому отримане число не можна інтерпретувати.

In [13]:
```
A, B = B['пара']
print('Ваша пара:', A, '×', B)
print()
print(f'{A}: шкала —', ПАСПОРТ.get(A, ('не у вашому списку',))[0] or '(визначте сам
print(f'{B}: шкала —', ПАСПОРТ.get(B, ('не у вашому списку',))[0] or '(визначте сам
print()
print(df[[A, B]].head(6).to_string())
```

```
Ваша пара: spO2 без дот × помер/жив

spO2 без дот: шкала — не у вашому списку
помер/жив: шкала — не у вашому списку

   spO2 без дот  помер/жив
0          90.0          1
1          83.0          2
2          78.0          2
3          82.0          2
4          80.0          2
5          72.0          1
```

In [14]:
```python
from scipy import stats

ОБРАНА_МІРА = 'точковий бісеріальний коефіцієнт'
ЧОМУ_САМЕ_ВОНА = ('spO2 є величиною зі шкали відношень, а помер/жив є бінарною номі
                  'тому для пари «відношень × бінарна номінальна» за таблицею потрі

assert ОБРАНА_МІРА and len(ЧОМУ_САМЕ_ВОНА) >= 40, 'Заповніть обидва поля'

# ваш код: обчисліть обрану міру
died = (df['помер/жив'] == 1).astype(int)                    # 1 = помер, 2 = жив
sat = df['spO2 без дот'].where(df['spO2 без дот'] > 20)      # 0,9 — помилка введ
pair = pd.DataFrame({'sat': sat, 'died': died}).dropna()
r_pb, p_pb = stats.pointbiserialr(pair['died'], pair['sat'])

print('n =', len(pair), '(0,9 виключено як помилку введення)')
print('точковий бісеріальний коефіцієнт r_pb = %.3f, p = %.2g' % (r_pb, p_pb))
print()
print(pair.groupby('died')['sat'].agg(['count', 'median', 'mean']).rename(index={0:
med_dead = pair.loc[pair['died'] == 1, 'sat'].median()
med_alive = pair.loc[pair['died'] == 0, 'sat'].median()
```

```
n = 119 (0,9 виключено як помилку введення)
точковий бісеріальний коефіцієнт r_pb = -0.438, p = 6.3e-07

       count  median  mean
died
жив       58    89.0  87.1
помер     61    80.0  78.6
```

In [15]:
```python
# Тепер — контрприклад: застосуйте кореляцію Пірсона незалежно від шкал
# і поясніть, чому отримане число не можна використовувати.

# 1) Пірсон на сирих кодах 1/2 для нашої пари: модуль той самий, але знак залежить
raw = pd.DataFrame({'код': df['помер/жив'], 'sat': sat}).dropna()
r_raw = raw['код'].corr(raw['sat'])
r_recoded = raw['код'].map({1: 2, 2: 1}).corr(raw['sat'])
print('Пірсон для кодів (1 = помер, 2 = жив): %.3f' % r_raw)
print('Пірсон після перекодування (1 = жив, 2 = помер): %.3f   <- знак змінився без

# 2) По-справжньому шкідливо — для номінальної змінної з кількома категоріями: Стат
sex_raw = df['Стать'].astype(str).str.strip()
code_a = sex_raw.map({'ж': 1, 'жін': 2, 'ч': 3, 'чол': 4})
code_b = sex_raw.map({'чол': 1, 'ч': 2, 'жін': 3, 'ж': 4})
```

```
r_a, r_b = code_a.corr(sat), code_b.corr(sat)
print()
print('Пірсон між кодом статі та spO2, кодування А: %.3f' % r_a)
print('Пірсон між кодом статі та spO2, кодування Б: %.3f   <- той самий стовпець, і
```

```
Пірсон для кодів (1 = помер, 2 = жив): 0.438
Пірсон після перекодування (1 = жив, 2 = помер): -0.438   <- знак змінився без жодно
ї зміни даних

Пірсон між кодом статі та spO2, кодування А: 0.023
Пірсон між кодом статі та spO2, кодування Б: -0.023   <- той самий стовпець, інше чи
сло
```

Етап 5. Підготовка змінних до моделі

Шкала визначає не лише статистику, а й спосіб кодування.

Шкала	Як готують до моделі
Номінальна	one-hot кодування; порядок категорій не вводять
Порядкова	впорядкована категорія або цілі коди із **заданим** порядком
Інтервальна	стандартизація; для дат — перехід до тривалостей
Відношень	нормалізація або стандартизація; можливе логарифмування
Ідентифікатор	у модель не беруть узагалі

Для кожного свого стовпця вкажіть спосіб підготовки — і **виконайте його** щонайменше для трьох стовпців різних шкал.

In [16]:
```
ПІДГОТОВКА = {
    # 'стовпець': 'спосіб і коротке обґрунтування',
    'номер і/х':      'у модель не беремо взагалі: це ідентифікатор, а не ознака',
    'Стать':          'спершу уніфікуємо написання (ж/жін, ч/чол), потім one-hot ко
    'ЛН':             'впорядкована категорія I < I-II < II < II-III < III < III-IV
    'Виписка':        'від дат переходимо до тривалості: кількість днів від поступл
    'Днів до поступ': 'витягаємо число з тексту, далі логарифм log1p і стандартизац
    'Т 37-37,5':      'витягаємо кількість днів з тексту, пропуски лишаємо як пропу
    'Гемогл2':        'стандартизація (z-оцінка), текст «немає» перетворюємо на про
    'Прот час1':      'логарифмування скошеного розподілу, потім стандартизація',
}
for c in МОЇ:
    ПІДГОТОВКА.setdefault(c, '')

pd.DataFrame([(c, ПАСПОРТ[c][0], ПІДГОТОВКА[c]) for c in МОЇ],
             columns=['Стовпець', 'Шкала', 'Як готуємо'])
```

Out[16]:

	Стовпець	Шкала	Як готуємо
0	номер і/х	ідентифікатор	у модель не беремо взагалі: це ідентифікатор, а не ознака
1	Стать	номінальна	спершу уніфікуємо написання (ж/жін, ч/чол), потім one-ho...
2	ЛН	порядкова	впорядкована категорія I < I-II < II < II-III < III < II...
3	Виписка	інтервальна	від дат переходимо до тривалості: кількість днів від пос...
4	Днів до поступ	відношень	витягаємо число з тексту, далі логарифм log1p і стандарт...
5	Т 37-37,5	відношень	витягаємо кількість днів з тексту, пропуски лишаємо як п...
6	Гемогл2	відношень	стандартизація (z-оцінка), текст «немає» перетворюємо на...
7	Прот час1	відношень	логарифмування скошеного розподілу, потім стандартизація

In [17]:

```
# ваш код: реалізуйте підготовку для трьох стовпців різних шкал

# номінальна: Стать -> one-hot
sex_onehot = pd.get_dummies(get_clean('Стать'), prefix='Стать').astype(int)

# порядкова: ЛН -> цілі коди із заданим порядком (I < I-II < ... < III-IV)
ln_coded = (get_clean('ЛН').cat.codes + 1).where(get_clean('ЛН').notna()).rename('Л

# інтервальна: Виписка -> тривалість госпіталізації в днях
stay_raw = (get_clean('Виписка') - pd.to_datetime(df['Поступлення'])).dt.days
n_bad_stay = int((stay_raw < 0).sum())                    # виписка раніше за пос
stay_days = stay_raw.where(stay_raw >= 0).rename('днів_у_стаціонарі')

# відношень: Гемогл2 -> стандартизація
hb = get_clean('Гемогл2')
hb_z = ((hb - hb.mean()) / hb.std()).rename('Гемогл2_z')

# відношень: Прот час1 -> логарифм і стандартизація
pt_log = np.log(get_clean('Прот час1'))
pt_z = ((pt_log - pt_log.mean()) / pt_log.std()).rename('Прот час1_logz')

prepared = pd.concat([sex_onehot, ln_coded, stay_days, hb_z, pt_z], axis=1)
print('підготовлено ознак:', prepared.shape[1], '| рядків:', prepared.shape[0])
print('тривалість стаціонару: від', int(stay_days.min()), 'до', int(stay_days.max()),
      '| виключено рядків, де виписка раніше за поступлення:', n_bad_stay)
prepared.describe().round(2).T
```

```
підготовлено ознак: 6 | рядків: 121
тривалість стаціонару: від 2 до 194 днів | виключено рядків, де виписка раніше за по
ступлення: 2
```

Out[17]:

	count	mean	std	min	25%	50%	75%	max
Стать_ж	121.0	0.62	0.49	0.00	0.00	1.00	1.00	1.00
Стать_ч	121.0	0.38	0.49	0.00	0.00	0.00	1.00	1.00
ЛН_код	121.0	3.57	1.20	1.00	3.00	4.00	4.00	6.00
днів_у_стаціонарі	116.0	19.65	20.90	2.00	10.00	16.00	22.25	194.00
Гемогл2_z	104.0	0.00	1.00	-3.30	-0.54	0.22	0.65	2.61
Прот час1_logz	112.0	-0.00	1.00	-1.94	-0.74	-0.21	0.58	3.16

Перед нормалізацією обов'язково перевірте варіативність. Стовпець, у якому всі значення однакові, дасть нуль у знаменнику — і мовчазний `NaN` замість помилки.

In [18]:
```
for c in МОЇ:
    з = pd.to_numeric(df[c], errors='coerce').dropna()
    if len(з) and з.nunique() == 1:
        print(f'УВАГА: {c} — сталий стовпець, нормалізація дасть ділення на нуль')
    elif len(з) and з.max() == з.min():
        print(f'УВАГА: {c} — max дорівнює min')
print('Перевірку варіативності виконано.')
```

```
Перевірку варіативності виконано.
```

In [19]:
```
# --- САМОПЕРЕВІРКА 2 ---
assert all(len(ПІДГОТОВКА[c].strip()) >= 15 for c in МОЇ), \
    'Для кожного стовпця вкажіть спосіб підготовки'
assert len(БЕЗГЛУЗДІ) >= 3, 'Етап 3 не завершено'
print('Самоперевірка 2 пройдена.')
```

```
Самоперевірка 2 пройдена.
```

Етап 6. Виправлення чужого аналізу

Відкрийте файл **`ЛР3_помилковий_аналіз.ipynb`**. Це звіт, який виконується без жодної помилки й містить десять хибних місць.

Для кожного з десяти заповніть рядок нижче:

- **розділ** — номер розділу звіту;
- **що не так** — яка шкала порушена або яке припущення хибне;
- **чому число неправильне** — що саме воно означає насправді;
- **як правильно** — коректний варіант або обґрунтована відмова від показника.

In [20]:
```
ВИПРАВЛЕННЯ = [
    # (розділ, 'що не так', 'чому число неправильне', 'як правильно'),
    ('1. Результат лікування',
```

```
     'Середнє від коду номінальної змінної: 1 = помер, 2 = жив.',
     'Коди 1 і 2 лише мітки двох категорій. Число 1,50 каже тільки, що померлих (61
     'Лише частоти й частки: померли 61 (50,4%), живі 60 (49,6%), мода — «помер».')
    ('2. Демографія',
     'Нечисті категорії: те саме значення записано по-різному (ж/жін, ч/чол).',
     'Вісім категорій нічого не говорять про «чотири статі»: це два значення в чоти
     'Спершу уніфікувати написання, тоді рахувати: жінки 75 (62,0%), чоловіки 46 (3
    ('3. Тяжкість за КТ',
     'Порядкова шкала: сортування за алфавітом замість порядку тяжкості та середнє
     'sort_index упорядковує за літерами, тому «найлегшим» став «важкий». Коди 0…11
     'Очистити написання (легка, сердній, серед ступ…), задати порядок легкий < сер
    ('4. Обсяг ураження за КТ',
     'Результат отримано на даних, непридатних до обчислення: стовпець КТ містить і
     'Через errors=coerce 17 з 44 значень (39%) мовчки стали NaN, до того ж саме на
     'Розібрати діапазони (середина діапазону) і порахувати заново, чесно вказавши
    ('5. Сатурація',
     'Помилка введення (0,9%) у даних і середнє по змішаній групі.',
     'Значення 0,9% фізіологічно неможливе, воно тягне середнє вниз (82,05), а міні
     'Виключити (або перевірити) 0,9, використати медіану й міжквартильний розмах і
    ('6. Дати поступлення',
     'Коефіцієнт варіації для дат (інтервальна шкала).',
     'Нуль календаря умовний (1970 рік), тому CV залежить від точки відліку: 0,595%
     'Для дат рахувати різниці: розмах у днях, стандартне відхилення в днях, кількі
    ('7. Стать і результат',
     'Кореляція Пірсона між двома номінальними змінними.',
     'Коди 0/1 для статі й результату довільні: знак r = 0,177 змінюється від перек
     'Таблиця спряженості, χ²-тест незалежності та коефіцієнт Крамера V (для таблиц
    ('8. Нормалізація',
     'Min-max нормалізація всіх числових стовпців без урахування шкали й варіативно
     'У вибірку потрапили код результату (номінальна змінна), стовпці майже без дан
     'Нормалізувати лише стовпці шкал відношень та інтервальної, спершу перевіряючи
    ('9. Очищення від пропусків',
     'df.dropna() для набору з 77 стовпців.',
     'Після видалення лишилось 0 рядків, а текст говорить про «повністю заповнений
     'Оцінити частку пропусків по стовпцях, відкинути майже порожні стовпці, а drop
    ('10. Рейтинг ознак',
     'Витік цільової змінної та кореляція Пірсона на непридатних даних.',
     'Дата розтину є лише у померлих (NaT перетворено на велике число), тож r = 1,0
     'Виключити ознаки, відомі лише після результату, брати тільки дані при надходж
]

assert len(ВИПРАВЛЕННЯ) == 10, f'Знайдено {len(ВИПРАВЛЕННЯ)} помилок із 10'
for p in ВИПРАВЛЕННЯ:
    assert len(p) == 4 and all(str(x).strip() for x in p), 'Заповніть усі чотири по
    assert len(str(p[2])) >= 30 and len(str(p[3])) >= 25, 'Пояснення закороткі'

pd.DataFrame(ВИПРАВЛЕННЯ, columns=['Розділ', 'Що не так', 'Чому число неправильне',
```

Out[20]:

	Розділ	Що не так	Чому число неправильне	Як правильно
0	1. Результат лікування	Середнє від коду номінальної змінної: 1 = помер, 2 = жив.	Коди 1 і 2 лише мітки двох категорій. Число 1,50 каже ті...	Лише частоти й частки: померли 61 (50,4%), живі 60 (49,6...
1	2. Демографія	Нечисті категорії: те саме значення записано по-різному ...	Вісім категорій нічого не говорять про «чотири статі»: ц...	Спершу уніфікувати написання, тоді рахувати: жінки 75 (6...
2	3. Тяжкість за КТ	Порядкова шкала: сортування за алфавітом замість порядку...	sort_index упорядковує за літерами, тому «найлегшим» ста...	Очистити написання (легка, сердній, серед ступ...), задати...
3	4. Обсяг ураження за КТ	Результат отримано на даних, непридатних до обчислення: ...	Через errors=coerce 17 з 44 значень (39%) мовчки стали N...	Розібрати діапазони (середина діапазону) і порахувати за...
4	5. Сатурація	Помилка введення (0,9%) у даних і середнє по змішаній гр...	Значення 0,9% фізіологічно неможливе, воно тягне середнє...	Виключити (або перевірити) 0,9, використати медіану й мі...
5	6. Дати поступлення	Коефіцієнт варіації для дат (інтервальна шкала).	Нуль календаря умовний (1970 рік), тому CV залежить від ...	Для дат рахувати різниці: розмах у днях, стандартне відх...
6	7. Стать і результат	Кореляція Пірсона між двома номінальними змінними.	Коди 0/1 для статі й результату довільні: знак r = 0,177...	Таблиця спряженості, χ^2-тест незалежності та коефіцієнт ...
7	8. Нормалізація	Min-max нормалізація всіх числових стовпців без урахуван...	У вибірку потрапили код результату (номінальна змінна), ...	Нормалізувати лише стовпці шкал відношень та інтервально...
8	9. Очищення від пропусків	df.dropna() для набору з 77 стовпців.	Після видалення лишилось 0 рядків, а текст говорить про ...	Оцінити частку пропусків по стовпцях, відкинути майже по...
9	10. Рейтинг ознак	Витік цільової змінної та кореляція Пірсона на непридатн...	Дата розтину є лише у померлих (NaT перетворено на велик...	Виключити ознаки, відомі лише після результату, брати ті...

Виправте щонайменше три помилки в коді

Оберіть три з десяти та покажіть правильне обчислення поруч із неправильним.

In [21]:
```
from scipy import stats
```

```
# --- розділ 1: замість середнього коду — частоти
outcome = df['помер/жив'].map({1: 'помер', 2: 'жив'})
print('РОЗДІЛ 1 (було: середнє 1.5)')
print(pd.DataFrame({'осіб': outcome.value_counts(), '%': (100 * outcome.value_count
print()

# --- розділ 2: уніфікуємо написання статі
print('РОЗДІЛ 2 (було: 4 статі)')
print(pd.DataFrame({'осіб': get_clean('Стать').value_counts(), '%': (100 * get_clea
print()

# --- розділ 3: ступінь КТ у правильному порядку
CT_MAP = {'легка': 'легкий', 'легкий': 'легкий', 'сердній': 'середній', 'серед ступ
          'серед ступеня': 'середній', 'середній': 'середній', 'важкий': 'важкий',
          'вкрай важкий': 'вкрай важкий', 'критичний': 'критичний'}
CT_ORDER = ['легкий', 'середній', 'важкий', 'вкрай важкий', 'критичний']
ct_label = df['ступінь КТ'].dropna().astype(str).str.strip().map(CT_MAP)
ct_cat = pd.Series(pd.Categorical(ct_label, categories=CT_ORDER, ordered=True))
print('РОЗДІЛ 3 (було: «найлегший — важкий», середнє «легкий»)')
print(ct_cat.value_counts().reindex(CT_ORDER).to_string())
print('медіана:', CT_ORDER[int(np.median(ct_cat.cat.codes))], '| нерозпізнаних:', i
print()

# --- розділ 4: КТ — розбираємо діапазони замість викидання
def parse_ct(v):
    if isinstance(v, (int, float)) and not pd.isna(v):
        return float(v)
    nums = [float(x.replace(',', '.')) for x in re.findall(r'\d+(?:[.,]\d+)?', str(
    return np.mean(nums) / 100 if nums else np.nan          # діапазон -> середина;

kt_raw = df['КТ'].dropna()
kt_old = pd.to_numeric(kt_raw, errors='coerce')
kt_new = kt_raw.map(parse_ct)
print('РОЗДІЛ 4 (було: 27 значень з 44, середнє 39.6%)')
print('значень було %d -> стало %d | середнє %.1f%% -> %.1f%% | медіана %.1f%% -> %
    kt_old.notna().sum(), kt_new.notna().sum(), 100 * kt_old.mean(), 100 * kt_new.m
    100 * kt_old.median(), 100 * kt_new.median()))
print()

# --- розділ 5: сатурація без помилки введення
sat_all = df['spO2 без дот']
print('РОЗДІЛ 5 (було: середнє 82.05, мінімум 0.9)')
print('з 0,9: середнє %.2f, мінімум %.1f | без 0,9: середнє %.2f, медіана %.1f, Q1-
    sat_all.mean(), sat_all.min(), sat.mean(), sat.median(), sat.quantile(.25), sat
print('медіана за результатом: помер %.1f | жив %.1f' % (med_dead, med_alive))
print()

# --- розділ 7: χ² та V замість Пірсона
table = pd.crosstab(get_clean('Стать'), outcome)
chi2, p_chi, dof, _ = stats.chi2_contingency(table, correction=False)
cramer_v = np.sqrt(chi2 / (table.values.sum() * (min(table.shape) - 1)))
print('РОЗДІЛ 7 (було: Пірсон 0.177)')
print(table.to_string())
print('χ² = %.2f, p = %.3f, V = %.3f' % (chi2, p_chi, cramer_v))
print()
```

```
# --- розділ 9: пропуски по стовпцях замість df.dropna()
miss = df.isna().mean()
work_cols = ['Вік', 'Стать', 'spO2 без дот', 'помер/жив']
print('РОЗДІЛ 9 (було: dropna() -> 0 рядків)')
print('стовпців з >50%% пропусків: %d із %d | df.dropna(): %d рядків | dropna(subse
    (miss > .5).sum(), df.shape[1], len(df.dropna()), work_cols, len(df.dropna(subs
```

```
РОЗДІЛ 1 (було: середнє 1.5)
           осіб     %
помер/жив
помер        61  50.4
жив          60  49.6

РОЗДІЛ 2 (було: 4 статі)
       осіб     %
Стать
ж        75  62.0
ч        46  38.0

РОЗДІЛ 3 (було: «найлегший — важкий», середнє «легкий»)
легкий           7
середній        18
важкий          10
вкрай важкий     4
критичний        3
медіана: середній | нерозпізнаних: 0

РОЗДІЛ 4 (було: 27 значень з 44, середнє 39.6%)
значень було 27 -> стало 43 | середнє 39.6% -> 42.5% | медіана 30.0% -> 35.0%

РОЗДІЛ 5 (було: середнє 82.05, мінімум 0.9)
з 0,9: середнє 82.05, мінімум 0.9 | без 0,9: середнє 82.73, медіана 84.0, Q1-Q3 80–8
9
медіана за результатом: помер 80.0 | жив 89.0

РОЗДІЛ 7 (було: Пірсон 0.177)
помер/жив  жив  помер
Стать
ж           32     43
ч           28     18
χ² = 3.78, p = 0.052, V = 0.177

РОЗДІЛ 9 (було: dropna() -> 0 рядків)
стовпців з >50% пропусків: 11 із 77 | df.dropna(): 0 рядків | dropna(subset=['Вік',
'Стать', 'spO2 без дот', 'помер/жив']): 120 рядків
```

Етап 7. Висновки та індивідуальне питання

In [22]:
```
ВИСНОВКИ = [
    # три висновки, кожен — повне речення про ваші дані, а не загальна фраза
    f'Сатурація без дотації пов’язана з результатом: медіана spO2 у померлих {med_d
```

```
    f'Файл потребує очищення перед будь-якою статистикою: у «Стать» два значення за
    'Шкала визначає допустимі підрахунки: для номера історії хвороби можна лише рах
]
assert len(ВИСНОВКИ) >= 3 and all(len(в) >= 60 for в in ВИСНОВКИ), \
    'Потрібні три змістовні висновки'
for i, в in enumerate(ВИСНОВКИ, 1):
    print(f'{i}. {в}')
```

1. Сатурація без дотації пов’язана з результатом: медіана spO2 у померлих 80%, у жив их 89%, точковий бісеріальний коефіцієнт r = -0.44 (знак мінус, бо помер = 1: чим ни жча сатурація, тим частіше летальний результат; p < 0,001, n = 119).
2. Файл потребує очищення перед будь-якою статистикою: у «Стать» два значення записа но чотирма способами, у «КТ» 17 з 44 значень текстові діапазони, у spO2 є помилка 0, 9, а df.dropna() залишає 0 рядків.
3. Шкала визначає допустимі підрахунки: для номера історії хвороби можна лише рахува ти частоти, для ЛН дозволені мода й медіана, для дат різниці в днях, а середнє, стан дартне відхилення та коефіцієнт варіації мають сенс тільки для шкали відношень (Днів до поступ, Гемогл2, Прот час1).

In [23]:
```
print('ВАШЕ ПИТАННЯ:'); print(B['питання']); print()

# числові докази: що означає порожня клітинка в «Дотація»
empty = df['Дотація'].isna()
alive = df['помер/жив'] == 2
n_empty = int(empty.sum())
n_empty_alive = int((empty & alive).sum())
sat_empty, sat_filled = sat[empty].median(), sat[~empty].median()
sat_min_empty = sat[empty].min()
alive_total, alive_kept = int(alive.sum()), int((~empty & alive).sum())
mean_all, mean_kept = sat.mean(), sat[~empty].mean()
dead_total = int((~alive).sum())
compl_dead = int(df.loc[~alive, 'Ускладнення'].notna().sum())
compl_alive = int(df.loc[alive, 'Ускладнення'].notna().sum())

ВІДПОВІДЬ = (
    f'Це стовпець «Дотація» (дні кисневої підтримки). Порожніх клітинок {n_empty},
    f'Сатурація без дотації в цих пацієнтів нормальна: медіана {sat_empty:.0f}%, мі
    f'Отже, порожньо означає не «невідомо», а «підтримка не потрібна», тобто норма.
    f'dropna() небезпечний, бо мовчки викидає саме цих найздоровіших пацієнтів: із
    f'Схожа ситуація в «Ускладнення»: воно заповнене у {compl_dead} з {dead_total}
)
assert len(ВІДПОВІДЬ.strip()) >= 200, 'Відповідь має бути розгорнутою (від 200 симв
print(ВІДПОВІДЬ)
```

ВАШЕ ПИТАННЯ:
Знайдіть у файлі стовпець, у якому порожня клітинка означає не «невідомо», а «норм а». Доведіть це числами й поясніть, чому dropna() тут небезпечний.

Це стовпець «Дотація» (дні кисневої підтримки). Порожніх клітинок 19, і всі вони нал ежать до живих (19 з 19). Сатурація без дотації в цих пацієнтів нормальна: медіана 9 5%, мінімум 89%, а в тих, у кого дотація заповнена, медіана лише 83%. Отже, порожньо означає не «невідомо», а «підтримка не потрібна», тобто норма. dropna() небезпечний, бо мовчки викидає саме цих найздоровіших пацієнтів: із 60 живих лишається 41, а сере дня spO2 штучно падає з 82.7 до 80.6, і вибірка зміщується в бік важких хворих. Схож а ситуація в «Ускладнення»: воно заповнене у 61 з 61 померлих і в 0 з 60 живих, тож порожнє означає «ускладнень не було».

Етап 8. Декларація про використання інструментів ШІ

Користуватися мовними моделями дозволено. Це нормальна інженерна практика.

Обов'язковою є прозорість: ви вказуєте, де саме і як скористалися інструментом — так само, як у науковій статті вказують використане програмне забезпечення. Задекларована допомога порушенням не є.

Порушенням є інше: здати роботу, яку ви не можете пояснити. На захисті вас попросять показати конкретний рядок свого коду, пояснити, чому він саме такий, і внести в нього невелику зміну на місці.

Для кожного етапу оберіть одне з формулювань:

- `'ні'` — інструментами ШІ не користувався
- `'синтаксис'` — питав про назву функції або синтаксис, код писав сам
- `'код, перевірено'` — код згенеровано, я його прочитав, перевірив і розумію
- `'текст, перероблено'` — згенеровано чернетку тексту, яку я переписав по суті
- `'текст, як є'` — згенерований текст залишено майже без змін

In [26]:
```
ДЕКЛАРАЦІЯ = {
    'етап 1 — паспорт змінних':        'код, перевірено',
    'етап 2 — допустимі статистики':   'код, перевірено',
    'етап 3 — безглузді числа':        'код, перевірено',
    'етап 4 — міра зв’язку':           'код, перевірено',
    'етап 5 — підготовка до моделі':   'код, перевірено',
    'етап 6 — виправлення аналізу':    'код, перевірено',
    'етап 7 — висновки й відповідь':   'код, перевірено',
}

ЩО_САМЕ_Я_ПЕРЕВІРИВ = 'Увесь код даної лабораторної перевірено, усі рядки зрозумілі
ПІДТВЕРДЖУЮ = True     # я можу пояснити кожен рядок свого коду
                       # і внести в нього зміни на вимогу під час захисту
```

In [27]:
```
# --- ФІНАЛЬНА САМОПЕРЕВІРКА ---
ФОРМУЛЮВАННЯ = {'ні', 'синтаксис', 'код, перевірено', 'текст, перероблено', 'текст,

перевірки = {
    'заповнено титульні поля':        bool(ПРІЗВИЩЕ_ІМЯ and ГРУПА and НОМЕР),
    'етап 1: паспорт із 8 стовпців':  all(ПАСПОРТ[c][0] and ПАСПОРТ[c][1] for c in
    'етап 3: три безглузді числа':    len(БЕЗГЛУЗДІ) >= 3,
    'етап 4: міру обрано й пояснено': bool(ОБРАНА_МІРА and len(ЧОМУ_САМЕ_ВОНА) >= 4
    'етап 5: підготовку описано':     all(len(ПІДГОТОВКА[c].strip()) >= 15 for c in
    'етап 6: знайдено 10 помилок':    len(ВИПРАВЛЕННЯ) == 10,
    'етап 7: три висновки':           len(ВИСНОВКИ) >= 3,
    'етап 7: відповідь на питання':   len(ВІДПОВІДЬ.strip()) >= 200,
```

```
    'декларацію заповнено':           all(v in ФОРМУЛЮВАННЯ for v in ДЕКЛАРАЦІЯ.val
    'опис перевірки ≥ 120 символів':  len(ЩО_САМЕ_Я_ПЕРЕВІРИВ.strip()) >= 120,
    'підтвердження авторства':        ПІДТВЕРДЖУЮ is True,
}

print('ЧЕК-ЛИСТ ГОТОВНОСТІ')
print('=' * 52)
for назва, ок in перевірки.items():
    print(('  ГОТОВО     ' if ок else '  НЕ ГОТОВО ') + назва)
print('=' * 52)

assert all(перевірки.values()), 'Робота не готова до здачі — див. чек-лист вище'
print()
print(f'Робота варіанта {В["варіант"]} готова до захисту.')
print(f'{ПРІЗВИЩЕ_ІМЯ}, {ГРУПА}')
```

```
ЧЕК-ЛИСТ ГОТОВНОСТІ
====================================================
  ГОТОВО     заповнено титульні поля
  ГОТОВО     етап 1: паспорт із 8 стовпців
  ГОТОВО     етап 3: три безглузді числа
  ГОТОВО     етап 4: міру обрано й пояснено
  ГОТОВО     етап 5: підготовку описано
  ГОТОВО     етап 6: знайдено 10 помилок
  ГОТОВО     етап 7: три висновки
  ГОТОВО     етап 7: відповідь на питання
  ГОТОВО     декларацію заповнено
  ГОТОВО     опис перевірки ≥ 120 символів
  ГОТОВО     підтвердження авторства
====================================================

Робота варіанта 19 готова до захисту.
Яцишин Роман, ШІ-32
```

Перед здачею

Виконайте **Kernel → Restart & Run All**. Ноутбук має пройти згори донизу без жодної помилки — це перше, що перевіряється на захисті.